

Student

I've been ignoring NaN values and duplicates in my DataFrames because I don't know how to handle them properly. Is that a big deal?

ChatGPT • AI Assistant

CONCEPT CARD

It is one of the biggest deals in data engineering. Missing data and duplicates are the two most common sources of **silent bugs** — your pipeline runs without errors, your model trains without errors, but your results are quietly wrong. A single NaN propagating through an aggregation makes the whole column NaN. A duplicate customer row inflates revenue metrics by 2×. Every professional data pipeline has explicit steps to detect, understand, and handle both.

This section covers the full lifecycle: detecting NaN and **None**, understanding how Pandas stores them, all four strategies for handling missing values with concrete guidance on when each is appropriate, type coercion problems in mixed-type columns, and deduplication with the critical **subset** and **keep** parameters.

EXAMPLE CARD

Dataset setup — a deliberately messy retail dataset:

python — Code Example

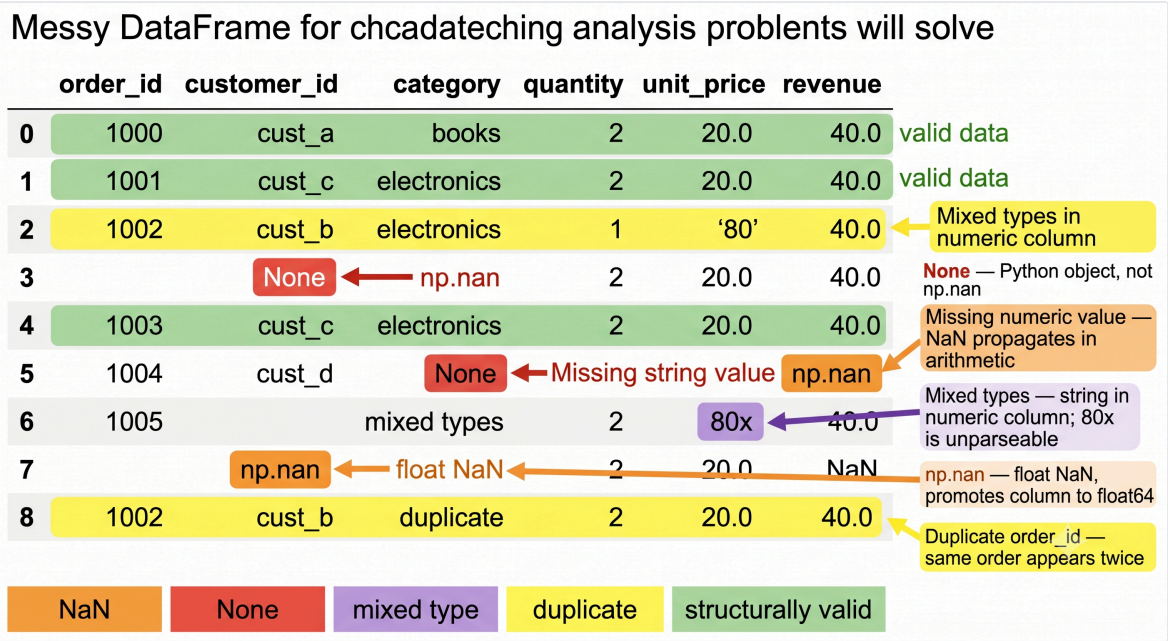
```
import pandas as pd
import numpy as np

# Messy orders dataset with realistic data quality issues
orders_messy = pd.DataFrame({
    'order_id': [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1002,
1008],
    'customer_id': [201, 202, None, 203, 202, 204, np.nan, 202,
201],
    'category': ['Electronics', 'Clothing', 'Books', 'Electronics',
None, 'Clothing', 'Books', 'Clothing',
'Electronics'],
    'quantity': [2, 1, 3, 1, 2, 4, 1, 1,
2],
    'unit_price': [500, '80', 25, 500, 80, '80x', 25, 80,
```

```
600],
    'revenue': [1000, 80, 75, 500, np.nan, 320, 25, 80,
1200]
})

print(orders_messy)
print('\nShape:', orders_messy.shape) # (9, 6)
print('\nDtypes:\n', orders_messy.dtypes)
```

Notice the issues: `None` vs `np.nan` in `customer_id`, a missing category, mixed types in `unitprice` (`'80'` and `'80x'` as strings alongside integers), a missing revenue, and a duplicate `order_id` `1002` .



Data Quality Issues Map for the Messy Orders Dataset

Data Quality Issue Types and Their Handling Strategies			
Issue Type	How It Appears	Detection Method	Fix Strategy
np.nan in numeric col	NaN in float/int column	.isna().sum()	dropna / fillna / interpolate
None in object col	None in string column	.isna().sum()	fillna with string constant
Integer promoted to	int64 becomes	.dtypes	Fill NaN, cast back to int

Issue Type	How It Appears	Detection Method	Fix Strategy
float	float64		
Mixed-type column	'80' and 80 in same col	.dtype == object	pd.to_numeric(errors='coerce')
Unparseable value	'80x' in numeric col	pd.to_numeric errors	coerce to NaN, then fill
Exact duplicate row	All columns identical	.duplicated().sum()	drop_duplicates()
Key duplicate	Same order_id, diff data	.duplicated(subset=[key])	drop_duplicates(subset, keep)

NP.NAN VS NONE: HOW PANDAS STORES MISSING VALUES

ChatGPT • AI Assistant

A two-column diagram. Left column labeled 'np.nan': a box showing 'type: float | IEEE 754 bit pattern: all exponent bits set + mantissa ≠ 0 | np.nan == np.nan → False'. Below it, an arrow pointing to 'Numeric column (int64)' with label 'Inserted into int column → entire column promoted to float64'. The int64 column on the left transforms with an arrow to a float64 column on the right, values changing from [1, 2, 3] to [1.0, 2.0, NaN]. Right column labeled 'None': a box showing 'type: NoneType | Python singleton | None == None → True'. Below it, two branches: Branch 1 arrow to 'object column (strings)' — 'None stored as Python object directly'; Branch 2 arrow to 'numeric column' — 'Pandas converts None → np.nan → float64 promotion'. At the bottom, a green box labeled 'Correct detection: always use .isna() not == np.nan'. A red box shows '== np.nan → always False, even on NaN values'. Purpose: visually explain the type mechanics that cause subtle bugs when developers assume None and np.nan are interchangeable.

np.nan vs None: Type Hierarchy and Pandas Storage Behavior

Student

I see both np.nan and None used for missing values. Are they the same thing?

Can I use them interchangeably?

ChatGPT • AI Assistant

CONCEPT CARD

They are different objects that Pandas treats similarly in most operations — but their differences cause real bugs if you don't understand them:

- ``np.nan`` is a **float** (`type(np.nan)` → `float`). It is the IEEE 754 floating-point "not a number" value. When you introduce `np.nan` into an integer column, Pandas **promotes the entire column to float64** because integers cannot represent NaN in NumPy's memory model.
- ``None`` is a **Python object** (`type(None)` → `NoneType`). In an `object`-dtype column (strings), Pandas stores `None` directly. In a numeric column, Pandas converts `None` to `np.nan` automatically.

EXAMPLE CARD

Example 1 — Basic: Type behavior of NaN and None

python — Code Example

```
import pandas as pd
import numpy as np

# np.nan is a float
print(type(np.nan))      # <class 'float'>
print(np.nan == np.nan)  # False ← critical: NaN is not equal to itself!
print(np.nan is np.nan)  # True ← but identical object

# None is NoneType
print(type(None))        # <class 'NoneType'>
print(None == None)      # True

# How Pandas handles each in a numeric column
s_int = pd.Series([1, 2, 3])
print(s_int.dtype)       # int64

s_nan = pd.Series([1, 2, np.nan]) # NaN forces float promotion
print(s_nan.dtype)       # float64 ← integers became floats!

s_none = pd.Series([1, 2, None])  # None also forces float promotion in numeric
```

```
print(s_none.dtype)    # float64
print(s_none)          # [1.0, 2.0, NaN] ← None became NaN
```

Student

That's surprising — why does adding one NaN turn all integers into floats?

ChatGPT • AI Assistant

It is a NumPy memory model constraint. NumPy arrays store data in contiguous typed memory blocks. An `int64` block stores 8-byte integers. There is no valid 8-byte integer bit pattern reserved for 'missing'. IEEE 754 float64 reserves specific bit patterns for NaN, so Pandas uses float64 as the nullable numeric type. This means:

- A column of IDs (e.g., `customer_id`) that should be integers becomes float after a single missing value
- When you print, you see `201.0` instead of `201` — cosmetically ugly and potentially causing issues in comparisons

EXAMPLE CARD

Example 2 — Intermediate: Integer promotion and how to check

python — Code Example

```
# customer_id column in our messy dataset has None and np.nan
print(orders_messy['customer_id'].dtype)    # float64 — promoted!
print(orders_messy['customer_id'])          # [201.0, 202.0, NaN,
203.0, ... ]

# Comparison trap: == np.nan ALWAYS returns False
print(orders_messy['customer_id'] == np.nan) # All False — useless!

# Correct way: use .isna() or .isnull() (they are identical)
print(orders_messy['customer_id'].isna())   # [False, False, True,
False, ... ]
```

⚠ WARNING

Never use `== np.nan` to check for missing values. Because `np.nan ≠ np.nan` by IEEE 754 definition, this comparison always returns `False` for every element. Always use `.isna()` or `pd.isna(value)` for scalar checks.

Example 3 — Practical: Pandas nullable integer dtype (modern fix)

python — Code Example

```
# Modern Pandas (1.0+) has nullable integer dtype: Int64 (capital I)
s_nullable = pd.array([1, 2, None], dtype='Int64') # Capital I =
nullable
print(s_nullable)          # <IntegerArray> [1, 2, <NA>]
print(s_nullable.dtype)    # Int64 (not float64!)

# In a DataFrame:
df = pd.DataFrame({'customer_id': pd.array([201, 202, None],
dtype='Int64')})
print(df['customer_id'].dtype) # Int64
print(df['customer_id'])      # [201, 202, <NA>] ← stays as integer
display
```

Student

Should I always use nullable Int64?

ChatGPT • AI Assistant

For new pipelines yes — it prevents the cosmetic and comparison issues of float promotion. However, most legacy code uses float64 with NaN, and many NumPy/scikit-learn functions still expect float64. Be pragmatic: if an ID column is losing its integer appearance due to NaN, use `Int64`. For numeric computation columns (revenue, price), float64 with NaN is still standard.

❗ COMMON PITFALL / MISTAKE

Incorrect — comparing to NaN:

python — Code Example

```
missing_mask = orders_messy['revenue'] == np.nan # All False — wrong!
print(missing_mask.sum()) # 0 — no missing detected
```

Correct — use `.isna()`:

python — Code Example

```
missing_mask = orders_messy['revenue'].isna() # Correct boolean
mask
```

```
print(missing_mask.sum()) # 1 - correctly detects 1 missing revenue
```

INTERVIEW CARD

- **Frequency:** High — asked in data engineering and ML engineering interviews
- **Question:** What is the difference between `np.nan` and `None` in a Pandas DataFrame? What happens to an integer column when you insert `np.nan` ?
- **Expected Answer:** `np.nan` is a float (IEEE 754 NaN); `None` is Python's null object. Inserting either into an integer column causes Pandas to promote the column dtype to `float64` because NumPy's `int64` has no NaN representation. The fix is Pandas nullable `Int64` dtype.
- **Follow-up:** Why does `df[df['col'] == np.nan]` always return an empty DataFrame?

PRACTICE QUIZ

1. What does `type(np.nan)` return?
2. What dtype does `pd.Series([1, 2, None])` have?
3. Why does `np.nan == np.nan` return `False` ?
4. Predict: `pd.Series([1, np.nan, 3]).sum()` — what is the result?
5. What is the modern Pandas dtype for nullable integers?
6. Debug: `df[df['id'] == np.nan]` returns 0 rows but you know there are missing IDs — what is wrong?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. `<class 'float'>`
2. `float64` — None in numeric Series triggers float promotion
3. IEEE 754 standard: NaN is defined as not equal to anything, including itself
4. `4.0` — `np.nan` is skipped by `.sum()` by default (`skipna=True`)
5. `Int64` (capital I) — `pd.Int64Dtype()`
6. `== np.nan` always returns False; use `.isna()` instead

CHECKPOINT QUESTION

A colleague writes `df['customer_id'].fillna(0)` to fix a column that was promoted to `float64` due to NaN. What two problems does this introduce, and what is

a better approach?

DETECTING AND VISUALISING MISSINGNESS

ChatGPT • AI Assistant

A three-panel diagram. Panel 1 labeled 'MCAR — Missing Completely At Random': shows a 6-row column with 2 randomly placed red cells (missing values), no pattern. Below: 'Cause: system error, random dropout. Test: missingness uncorrelated with any column. Fix: safe to drop rows or fill with mean/median.' Panel 2 labeled 'MAR — Missing At Random': shows a 6-row column where 2 missing values align with rows where another column shows 'signup=phone'. A dashed arrow connects the two columns. Below: 'Cause: depends on another observed column. Test: missingness correlates with observed column. Fix: group-based imputation or model imputation.' Panel 3 labeled 'MNAR — Missing Not At Random': shows a 6-row column where 2 missing values align with rows that would have been the highest values (annotated with dashed predicted values). Below: 'Cause: the value itself causes it to be missing. Test: no observed column explains it. Fix: do not fill with mean — acknowledge bias or model the mechanism.' A legend across the bottom shows risk levels: MCAR = low risk, MAR = medium risk, MNAR = high risk (red). Purpose: help students visually distinguish the three missingness types before choosing a fill strategy.

Three Types of Missingness and Their Imputation Implications

Student

Before I fix missing data, I need to understand how much is missing and where.
What tools does Pandas have for that?

ChatGPT • AI Assistant

CONCEPT CARD

The detection toolkit has three levels:

1. **Column-level summary** — how many missing per column and what percentage
2. **Row-level detection** — which rows have any missing value
3. **Pattern visualisation** — which columns are missing together (correlation of missingness)

Understanding the pattern matters because the *mechanism* of missingness determines the right fix: random missing values (safe to fill with mean), systematic missing (a missing value means something specific — filling with mean distorts analysis), or structurally missing (a value simply doesn't exist for that entity).

EXAMPLE CARD

Example 1 — Basic: Column-level missing summary

python — Code Example

```
import pandas as pd
import numpy as np

# Missing count per column
print(orders_messy.isna().sum())
# order_id      0
# customer_id    2
# category       1
# quantity       0
# unit_price     0
# revenue        1

# As percentage of total rows
missing_pct = orders_messy.isna().mean() * 100
print(missing_pct.round(1))
# customer_id    22.2%
# category       11.1%
# revenue        11.1%
```

Example 2 — Intermediate: Row-level detection

python — Code Example

```
# Which rows have ANY missing value
has_missing = orders_messy.isna().any(axis=1)
print(orders_messy[has_missing])
# Returns rows 2, 4, 5, 6 — the ones with NaN in any column

# Which rows have ALL values missing (completely empty rows)
all_missing = orders_messy.isna().all(axis=1)
print(all_missing.sum()) # 0 in our dataset

# Count missing per row
orders_messy['missing_count'] = orders_messy.isna().sum(axis=1)
print(orders_messy[['order_id', 'missing_count']])
```

Example 3 — Practical: Full missing data audit function

python — Code Example

```
def missing_audit(df):
    """Returns a tidy DataFrame summarising missingness per column."""
    audit = pd.DataFrame({
```

```

        'dtype':      df.dtypes,
        'count':      df.isna().sum(),
        'pct_missing': (df.isna().mean() * 100).round(2),
        'n_unique':    df.nunique(dropna=True)
    })
    return audit[audit['count'] > 0].sort_values('pct_missing',
ascending=False)

print(missing_audit(orders_messy))
#           dtype  count  pct_missing  n_unique
# customer_id float64    2         22.22         4
# category     object    1         11.11         4
# revenue      float64    1         11.11         7

```

Example 4 — Real Business: Heatmap-style text visualisation

python – Code Example

```

# Text-based missingness heatmap (without matplotlib)
def missing_heatmap_text(df):
    """Print a simple ASCII heatmap of missing values."""
    print('\nMissingness Map (■ = missing, . = present):')
    print('col'.ljust(15), ' '.join([str(i) for i in range(len(df))]))
    for col in df.columns:
        marks = ['■' if v else '.' for v in df[col].isna()]
        print(col.ljust(15), ' '.join(marks))

missing_heatmap_text(orders_messy)
# Missingness Map (■ = missing, . = present):
# col           0 1 2 3 4 5 6 7 8
# order_id      . . . . . . . .
# customer_id    . . ■ . . . ■ .
# category       . . . . ■ . . .
# quantity       . . . . . . . .
# unit_price     . . . . . . . .
# revenue        . . . . ■ . . .

```

Student

How do I know if the missing values are random or systematic?

There are three types of missingness that statisticians distinguish:

- **MCAR (Missing Completely At Random):** No pattern — missingness is unrelated to any data. Safe to drop rows or fill with column statistics.
- **MAR (Missing At Random):** Missingness is related to other observed columns but not the missing value itself. E.g., high-value customers have missing email because they signed up via phone. Fill using a model or group-based imputation.
- **MNAR (Missing Not At Random):** The missing value itself drives its absence. E.g., very high revenue orders are missing revenue because the system caps large transactions. Filling with mean will bias your analysis.

In practice: always check if missingness correlates with other columns before deciding how to fill:

python – Code Example

```
# Check if missing revenue correlates with category
orders_messy['rev_missing'] =
orders_messy['revenue'].isna().astype(int)
print(orders_messy.groupby('category')['rev_missing'].mean())
# If all missing revenue is in one category → likely MAR or MNAR, not
MCAR
```

INTERVIEW CARD

- **Frequency:** Moderate-high — asked in senior data roles and ML interviews
- **Question:** What is the difference between MCAR, MAR, and MNAR missing data? Why does it matter for imputation?
- **Expected Answer:** MCAR = missingness is random, safe to fill with column stats. MAR = missingness depends on observed columns, use group-based imputation. MNAR = the value itself causes it to be missing — filling with mean will bias results; must model the mechanism or acknowledge the limitation.
- **Follow-up:** How would you programmatically check if missingness in one column correlates with values in another column?

PRACTICE QUIZ

1. What does `df.isna().sum()` return?
2. What does `df.isna().any(axis=1)` return?
3. What is the difference between `isna()` and `isnull()` ?

4. Predict: `pd.Series([1, np.nan, 3, np.nan]).isna().mean()` — what is the result?
5. In a missing audit, what does a high `pct_missing` (>50%) column suggest about dropping vs filling?
6. Debug: `df.dropna()` removes too many rows — what parameter limits it to specific columns?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. A Series with column names as index and count of NaN values as values
2. A boolean Series — True for rows that have at least one NaN in any column
3. They are identical aliases — `isnull` is kept for backwards compatibility
4. `0.5` — 2 out of 4 values are NaN
5. High missingness suggests either drop the column entirely or use a model-based imputation — filling with mean when 50%+ is missing means you are inventing most of the column
6. `df.dropna(subset=['col1', 'col2'])` — only drops rows missing in those specific columns

❓ **CHECKPOINT QUESTION**

You run `missing_audit(df)` and find that the `email` column is 65% missing, while `signup_method` shows that all missing emails come from rows where `signup_method == 'phone'`. Is this MCAR, MAR, or MNAR? What do you do about it?

HANDLING MISSING VALUES: DROPNA, FILLNA, INTERPOLATE, AND DATA LEAKAGE

ChatGPT • AI Assistant

A four-row diagram showing 8 time steps (hours 1–8) for a temperature sensor. Row 1 labeled 'Original': values shown as 22.1, NaN, NaN, 23.5, NaN, 24.0, NaN, 23.8 — NaN cells shaded grey. Row 2 labeled 'ffill (forward fill)': values 22.1, 22.1, 22.1, 23.5, 23.5, 24.0, 24.0, 23.8 — the filled cells shaded orange, with horizontal arrows pointing rightward from the last known value to the NaN cells, labelled 'carry last known'. Row 3 labeled 'bfill (backward fill)': values 22.1, 23.5, 23.5, 23.5, 24.0, 24.0, 23.8, 23.8 — filled cells shaded blue, with leftward arrows from the next known value. Row 4 labeled 'interpolate (linear)': values 22.1, 22.57, 23.03, 23.5, 23.75, 24.0, 23.9, 23.8 — filled cells shaded green, with a diagonal dashed line connecting the known anchor points. Below all rows, a decision guide: 'Use ffill when: last known value is correct (prices, statuses). Use bfill when: next known value is correct (future-known corrections). Use interpolate when: value changed smoothly between measurements (temperatures, signals).' Purpose: give students a visual side-by-side comparison so they can intuitively choose the right method.

ffill vs bfill vs interpolate on a Sensor Time Series

Student

So now that I can detect missing values, what are my options for handling them? I usually just do `dropna()` but I feel like that's lazy.

ChatGPT • AI Assistant

CONCEPT CARD

There are four strategies, and you choose based on the mechanism (MCAR/MAR/MNAR) and context:

1. `dropna()` — remove rows or columns with missing values. Right when: missingness is MCAR, rows are few, and the missing column is critical.
2. `fillna(value)` — replace NaN with a constant, the column mean, median, or mode. Right when: MCAR or MAR, and the column is numeric.
3. `fillna(method='ffill'/'bfill')` — forward-fill or backward-fill. Right when: time series data where 'carry the last known value' is semantically correct.
4. `interpolate()` — fill with linearly interpolated values between known points. Right when: smooth numeric time series (temperatures, stock prices, sensor readings).

And one critical rule that trips up nearly every beginner: **compute fill statistics on training data only — never on the full dataset before splitting.**

EXAMPLE CARD

Example 1 — Basic: `dropna()`

python — Code Example

```
import pandas as pd
import numpy as np
```

```
print('Before dropna:', orders_messy.shape) # (9, 6)

# Drop ANY row with a NaN
cleaned = orders_messy.dropna()
print('After dropna():', cleaned.shape)      # (5, 6) – 4 rows dropped

# Drop only rows missing in specific critical columns
cleaned_selective = orders_messy.dropna(subset=['customer_id',
'revenue'])
print('After dropna(subset):', cleaned_selective.shape) # (6, 6)

# Drop columns that are entirely NaN (safer column pruning)
orders_messy.dropna(axis=1, how='all') # Only drops fully-empty
columns

# Drop columns missing more than 50% of rows
threshold = len(orders_messy) * 0.5
orders_messy.dropna(axis=1, thresh=int(threshold))
```

Example 2 — Intermediate: fillna() with statistics

python – Code Example

```
# Fill missing revenue with column median
median_rev = orders_messy['revenue'].median()
orders_messy['revenue'] = orders_messy['revenue'].fillna(median_rev)
print(f'Filled revenue NaN with median: {median_rev}')

# Fill missing category with mode (most frequent value)
mode_cat = orders_messy['category'].mode()[0] # mode() returns a
Series
orders_messy['category'] = orders_messy['category'].fillna(mode_cat)

# Fill missing customer_id with a sentinel value (unknown customer)
orders_messy['customer_id'] = orders_messy['customer_id'].fillna(-1)

# Fill multiple columns at once with a dictionary
fill_values = {'revenue': median_rev, 'category': 'Unknown',
'customer_id': -1}
orders_messy_filled = orders_messy.fillna(fill_values)
print(orders_messy_filled.isna().sum()) # All zeros
```

Example 3 — Practical: ffill and bfill for time series

python – Code Example

```

# Simulate sensor data with gaps
sensor = pd.DataFrame({
    'timestamp': pd.date_range('2024-01-01', periods=8, freq='h'),
    'temperature': [22.1, np.nan, np.nan, 23.5, np.nan, 24.0, np.nan,
23.8]
}).set_index('timestamp')

print('Original:')
print(sensor)

# Forward fill: carry last known value forward
print('\nAfter ffill:')
print(sensor.ffmpeg())
# 22.1, 22.1, 22.1, 23.5, 23.5, 24.0, 24.0, 23.8

# Backward fill: use next known value
print('\nAfter bfill:')
print(sensor.bfill())
# 22.1, 23.5, 23.5, 23.5, 24.0, 24.0, 23.8, 23.8

# Limit fill to N consecutive NaNs (avoids filling long gaps)
print('\nffill with limit=1:')
print(sensor.ffmpeg(limit=1))
# 22.1, 22.1, NaN, 23.5, 23.5, 24.0, 24.0, 23.8
# Second consecutive NaN left unfilled

```

Student

When should I use `ffill` vs `interpolate` ?

ChatGPT • AI Assistant

Use `'ffill'` when the last known value is the best estimate — e.g., a device that reports its last reading when offline, or a price that doesn't change until the next tick. Use `'interpolate'` when the true value likely changed smoothly between the two known points — e.g., temperature, stock prices, or weight measurements. `ffill` is a step function; `interpolate` is a linear ramp.

Example 4 — Real Business: `interpolate()` for smooth numeric gaps

python — Code Example

```

# Linear interpolation fills NaN by linearly connecting neighbouring
values
print('\nAfter interpolate (linear):')
print(sensor.interpolate(method='linear'))
# Original: 22.1, NaN, NaN, 23.5, NaN, 24.0, NaN, 23.8
# Interpolated:
# 22.1
# 22.57 (22.1 + (23.5-22.1)/3 * 1)
# 23.03 (22.1 + (23.5-22.1)/3 * 2)
# 23.5
# 23.75 (23.5 + (24.0-23.5)/2 * 1)
# 24.0
# 23.9 (24.0 + (23.8-24.0)/2 * 1)
# 23.8

# Also available: 'polynomial', 'spline', 'time' (uses index datetime)
sensor.interpolate(method='time') # Uses actual time gaps, not just
row positions

```

Example 5 — Advanced: The data leakage trap

python – Code Example

```

# WRONG: computing fill statistic on full dataset before split
import numpy as np

# Simulate a revenue dataset
revenue = pd.Series([100, np.nan, 300, np.nan, 500, 200, np.nan, 400])
full_mean = revenue.mean() # 300.0 – includes patterns from all 8
rows
revenue_filled = revenue.fillna(full_mean)

# Then split – too late! The NaN rows in test set were filled with
# statistics that include the test set's own values → data leakage
train = revenue_filled[:5]
test = revenue_filled[5:]

# CORRECT: split first, compute stats on train only, apply to both
train_raw = revenue[:5]
test_raw = revenue[5:]

train_mean = train_raw.mean() # 300.0 – computed from train
only
train_filled = train_raw.fillna(train_mean) # Fill train
test_filled = test_raw.fillna(train_mean) # Apply SAME stat to test

```

```
print(f'Train mean (no leakage): {train_mean:.1f}')
print('Train filled:', train_filled.values)
print('Test filled (using train stat):', test_filled.values)
```

⚠ WARNING

Data leakage from imputation is one of the most common mistakes in ML pipelines. When you fill NaN with `df['col'].mean()` before a train/test split, the test set's NaN values are filled using a mean that includes the test data — the model has implicitly seen test data during training. In scikit-learn pipelines, always use `SimpleImputer` inside a `Pipeline` so the fit (mean computation) happens only on training data and `transform` applies the stored statistic to test data.

❗ COMMON PITFALL / MISTAKE

Incorrect — fill before split:

python — Code Example

```
df['revenue'].fillna(df['revenue'].mean(), inplace=True) # Leaks test data!
X_train, X_test = train_test_split(df)                  # Too late!
```

Correct — split first, then fit imputer on train only:

python — Code Example

```
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

X_train, X_test = train_test_split(df)           # Split first
imp = SimpleImputer(strategy='mean')             # Imputer object
imp.fit(X_train[['revenue']])                    # Fit on train only
X_train['revenue'] = imp.transform(X_train[['revenue']])
X_test['revenue'] = imp.transform(X_test[['revenue']]) # Apply same stat
```

🧑 INTERVIEW CARD

- **Frequency:** Very high — data leakage is the #1 topic in ML engineering interviews
- **Question:** Why is it wrong to fill missing values with `df['col'].mean()` before a train/test split? What is the correct approach?

- **Expected Answer:** Computing the mean on the full dataset before splitting means the test set's NaN values are filled using statistics that include the test data itself — the model indirectly sees the test set during training. The correct approach is to split first, compute fill statistics only on the training set, then apply those same statistics to the test set using scikit-learn's `SimpleImputer` inside a `Pipeline`.
- **Follow-up:** What is the difference between `fillna(method='ffill')` and `interpolate()`?

🔗 PRACTICE QUIZ

1. Which `dropna` parameter limits removal to rows missing in specific columns?
2. What does `sensor.ffill(limit=1)` do differently from `sensor.ffill()`?
3. When is `interpolate(method='time')` preferred over `interpolate(method='linear')`?
4. Predict: `pd.Series([1.0, np.nan, np.nan, 4.0]).interpolate()` — what are the filled values?
5. You have 10,000 rows and 200 are missing revenue. `dropna()` removes them all. Is this safe?
6. Debug: after `fillna(df['col'].mean())`, your model's cross-validation score is suspiciously high — what might be wrong?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `subset=['col1', 'col2']` parameter
2. `limit=1` only fills one consecutive NaN — a second consecutive NaN is left as-is. Without limit, all consecutive NaNs in a run are filled.
3. When the DatetimeIndex has irregular time gaps — `method='time'` weights interpolation by actual elapsed time, not row position
4. `[1.0, 2.0, 3.0, 4.0]` — linear interpolation divides the gap (4-1=3) equally over 2 steps
5. Generally safe if MCAR and 2% missing (200/10000) — small fraction unlikely to bias results significantly
6. Data leakage — the mean was computed on the full dataset (including validation/test folds) before cross-validation; use `SimpleImputer` inside a `Pipeline` so imputation is fit fresh on each training fold

❓ CHECKPOINT QUESTION

You have a 30-day temperature sensor dataset where days 10–15 are missing (6 consecutive days — the sensor was offline). Should you use `ffill`, `bfill`, or `interpolate`? What is the risk with each approach for this specific gap?

TYPE COERCION AND MIXED-TYPE COLUMNS

ChatGPT • AI Assistant

A three-column diagram. The input on the left is a Series with 5 values: 500 (int), '80' (string), 25 (int), '80x' (string — highlighted red), 600 (int). Three rightward arrows branch to three output columns. Column 1 labeled 'errors=raise': shows a red X symbol with label 'Raises ValueError immediately at 80x — entire operation fails'. Column 2 labeled 'errors=ignore': shows the original Series unchanged, with label 'Returns original object dtype — no conversion at all'. Column 3 labeled 'errors=coerce': shows [500.0, 80.0, 25.0, NaN, 600.0] — the '80x' row becomes NaN (shaded orange), all others converted to float. Below column 3, two sub-steps: 'Step 1: log df[result.isna()]' and 'Step 2: fillna(median) or drop'. A decision guide at bottom: 'raise → use for debugging (fail fast); ignore → almost never use; coerce → use in production pipelines with logging'. Purpose: make the three error modes concrete and show why coerce is the production-appropriate choice.

pd.to_numeric errors Parameter Behavior Comparison

Student

In our messy dataset, the `unit_price` column has both numbers and strings like `'80'` and `'80x'`. How do I handle that?

ChatGPT • AI Assistant

CONCEPT CARD

Mixed-type columns are one of the most common real-world data quality problems. They happen when:

- A CSV is exported with some cells containing text notes in a numeric column
- Different data sources are concatenated with inconsistent types
- A form allows free-text input in a numeric field

When Pandas reads a column where some values are strings and some are numbers, it assigns `dtype=object` — it stores everything as Python objects, turning off all numeric optimisations. The fix is `'pd.to_numeric(errors='coerce')'`, which converts parseable values to numbers and converts unparseable strings to `NaN`.

EXAMPLE CARD

Example 1 — Basic: Detecting mixed-type columns

python — Code Example

```

import pandas as pd
import numpy as np

print(orders_messy['unit_price'].dtype)    # object — mixed type!
print(orders_messy['unit_price'])

# 0      500
# 1       80    ← stored as string '80'
# 2       25
# 3      500
# 4       80
# 5     80x    ← unparseable string
# 6       25
# 7       80
# 8      600

# Check each value's actual Python type
print(orders_messy['unit_price'].apply(type).value_counts())
# <class 'int'>      7
# <class 'str'>      2

```

Example 2 — Intermediate: pd.to_numeric with errors='coerce'

python — Code Example

```

# errors='raise' — raises ValueError on first bad value (default)
# errors='ignore' — returns original object dtype unchanged
# errors='coerce' — converts bad values to NaN, good values to numeric

orders_messy['unit_price_clean'] = pd.to_numeric(
    orders_messy['unit_price'],
    errors='coerce'
)

print(orders_messy[['unit_price', 'unit_price_clean']])
# unit_price | unit_price_clean
# 500        | 500.0
# 80 (str)   | 80.0    ← parseable string → numeric
# 25        | 25.0
# 500       | 500.0
# 80        | 80.0
# 80x (str) | NaN     ← unparseable → NaN
# 25       | 25.0

```

```
# 80 (str) | 80.0
# 600      | 600.0

print(orders_messy['unit_price_clean'].dtype)    # float64
print(orders_messy['unit_price_clean'].isna().sum()) # 1 - the '80x'
row
```

Student

What happened to '80x'? It turned into NaN — should I investigate it or just drop it?

ChatGPT • AI Assistant

🌐 REAL WORLD APPLICATION

Always investigate before dropping. In production data engineering, a value like '80x' almost always signals a data entry error or a system bug — it might mean the true value is 80 (the 'x' is a typo), or it could flag a special pricing rule. The correct professional response is:

1. Log the unparseable values before coercing
2. Raise the issue with the data owner
3. Apply a business rule (e.g., strip non-numeric characters and retry)

python – Code Example

```
# Step 1: Log unparseable rows before coercing
bad_rows = orders_messy[pd.to_numeric(orders_messy['unit_price'],
errors='coerce').isna()]
print('Rows with unparseable unit_price:')
print(bad_rows[['order_id', 'unit_price']])
# order_id=1006, unit_price='80x'

# Step 2: Attempt to salvage – strip non-numeric chars
orders_messy['unit_price_salvaged'] = (
    orders_messy['unit_price']
    .astype(str)
    .str.extract(r'(\d+\.? \d*)') # Extract leading numeric part
    .squeeze()
)
orders_messy['unit_price_final'] = pd.to_numeric(
    orders_messy['unit_price_salvaged'], errors='coerce'
```

```
)
print(orders_messy[['order_id', 'unit_price', 'unit_price_final']])
# '80x' → extracted '80' → 80.0
```

Example 3 — Practical: Type coercion for multiple columns at once

python – Code Example

```
# Real pipeline pattern: apply to_numeric to all suspected numeric
columns
numeric_cols = ['unit_price', 'quantity', 'revenue']
for col in numeric_cols:
    original_nulls = orders_messy[col].isna().sum()
    orders_messy[col] = pd.to_numeric(orders_messy[col],
errors='coerce')
    new_nulls = orders_messy[col].isna().sum()
    coerced = new_nulls - original_nulls
    if coerced > 0:
        print(f'WARNING: {col} – {coerced} values coerced to NaN (were
unparseable strings)')

# After coercion, check dtypes
print(orders_messy[numeric_cols].dtypes)
# All should be float64
```

Example 4 — Real Business: Downcast after coercion for memory savings

python – Code Example

```
# After successful coercion, downcast to save memory
orders_messy['unit_price_final'] = pd.to_numeric(
    orders_messy['unit_price'], errors='coerce'
)
# Downcast: float64 → float32 if values fit, int64 → int32 etc.
orders_messy['unit_price_final'] = pd.to_numeric(
    orders_messy['unit_price_final'], downcast='float'
)
print(orders_messy['unit_price_final'].dtype) # float32 – half the
memory

# Or use integer downcast when no NaN remains
quantity_clean = pd.to_numeric(orders_messy['quantity'],
errors='coerce')
quantity_clean = quantity_clean.fillna(0).astype(int)
print(quantity_clean.dtype) # int64
```

```
# Downcast further
quantity_clean = pd.to_numeric(quantity_clean, downcast='integer')
print(quantity_clean.dtype) # int8 - values 0-4 fit in int8
```

❗ COMMON PITFALL / MISTAKE

Incorrect — silently ignoring coercion failures:

python — Code Example

```
df['price'] = pd.to_numeric(df['price'], errors='ignore') # Returns
object dtype unchanged!
df['price'].mean() # TypeError: unsupported operand type for strings
```

Correct — use coerce, then handle NaN:

python — Code Example

```
df['price'] = pd.to_numeric(df['price'], errors='coerce') # Bad
values → NaN
bad = df[df['price'].isna()] # Log the bad rows
df['price'] = df['price'].fillna(df['price'].median()) # Fill NaN
```

🧐 INTERVIEW CARD

- **Frequency:** Moderate — very common in data engineering and ETL roles
- **Question:** A numeric column in your DataFrame has dtype `object`. What steps do you take to convert it to a proper numeric type?
- **Expected Answer:** (1) Inspect with `.apply(type).value_counts()` to see what types are present. (2) Apply `pd.to_numeric(col, errors='coerce')` — parseable strings become numbers, unparseable become NaN. (3) Log and investigate the NaN rows. (4) Decide whether to drop, salvage (strip non-numeric characters), or fill. (5) Optionally downcast to save memory.
- **Follow-up:** What does `errors='ignore'` do in `pd.to_numeric`, and why is it almost always the wrong choice?

❓ PRACTICE QUIZ

1. What does `pd.to_numeric(s, errors='coerce')` do to the string `'abc'` ?
2. What dtype does a Pandas column become when it contains mixed int and str values?
3. What does `downcast='float'` do in `pd.to_numeric` ?

4. Predict: `pd.to_numeric(pd.Series(['1', '2.5', 'three', '4']), errors='coerce')` — what are the values?
5. How do you find all rows where a coercion introduced new NaN values?
6. Debug: `df['price'].mean()` raises `TypeError` even after `pd.to_numeric` — what likely happened?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. Converts it to `NaN` — the string cannot be parsed as a number
2. `object` dtype — Pandas stores all values as Python objects
3. Attempts to downcast the result to the smallest float type that can losslessly represent the values (e.g., `float64` → `float32`)
4. `[1.0, 2.5, NaN, 4.0]` — 'three' cannot be parsed, becomes NaN
5. Compare `isna().sum()` before and after coercion: `new_nans = after.isna().sum() - before.isna().sum()`
6. `errors='ignore'` was used instead of `errors='coerce'` — the column was returned unchanged as object dtype; fix by using `errors='coerce'`

❓ **CHECKPOINT QUESTION**

A CSV file has a `revenue` column where most rows are numbers but some rows contain the text `'N/A'`, `'-'`, or `''` (empty string). Write the complete two-step cleaning pipeline: first convert to numeric (handling all three cases), then fill any resulting NaN with the column median.

DEDUPLICATION: DUPLICATED, DROP_DUPLICATES, SUBSET, AND KEEP

ChatGPT • AI Assistant

A four-panel diagram, each showing a 5-row table where rows with `order_id=1002` are highlighted. The table columns are: `row_number` (1–5), `order_id` (1001, 1002, 1003, 1002, 1004), `updated_at` (dates). Panel 1 labeled 'Input (unsorted)': both 1002 rows highlighted yellow — positions 2 and 4. Panel 2 labeled 'keep=first': row 4 (second 1002) is shaded red with label 'MARKED AS DUPLICATE — will be dropped'. Row 2 (first 1002) is shaded green 'KEPT'. Panel 3 labeled 'keep=last': row 2 (first 1002) is shaded red 'MARKED', row 4 is green 'KEPT'. Panel 4 labeled 'keep=False': BOTH 1002 rows (2 and 4) are shaded red 'BOTH MARKED — both will be dropped'. Below the panels, a recommendation box: 'Best practice for update-aware dedup: sort by [key, timestamp] FIRST, then keep=last — guarantees the most recent record is retained regardless of original row order'. Purpose: make the keep parameter's effect on which row is marked concrete and visual before students write any code.

deduplicated() keep Parameter Behaviour Illustrated

How do I find and remove duplicate rows? Our orders dataset has `order_id=1002` appearing twice — I need to keep only the most recent one.

ChatGPT • AI Assistant

CONCEPT CARD

Pandas has two deduplication methods:

- `.duplicated(subset, keep)` — returns a boolean mask marking duplicate rows. Use it to **inspect** before removing.
- `.drop_duplicates(subset, keep)` — removes duplicate rows. Use it to **clean**.

Both share two critical parameters:

- `'subset'` — which columns to consider when defining 'duplicate'. Default is all columns (exact row match). Specify a list to deduplicate by a business key (e.g., `subset=['order_id']`).
- `'keep'` — which occurrence to keep: `'first'` (default), `'last'` (most recent in the DataFrame), or `False` (mark/drop ALL occurrences).

🔗 ANALOGY

Think of `duplicated()` as a highlighter — it marks which rows are duplicates. `drop_duplicates()` is the eraser — it removes the marked rows. Always highlight first to understand the problem, then erase.

EXAMPLE CARD

Example 1 — Basic: Detecting duplicates

python — Code Example

```
import pandas as pd
import numpy as np

# Our messy dataset has order_id 1002 appearing twice
print(orders_messy['order_id'].duplicated().sum()) # 1 duplicate key

# Full row exact-match duplicates
print(orders_messy.duplicated().sum()) # 0 — rows are NOT identical
# (same order_id but different row content → not a full-row duplicate)
```

```
# Duplicate by business key (order_id)
dup_mask = orders_messy.duplicated(subset=['order_id'], keep=False)
print(orders_messy[dup_mask][['order_id', 'customer_id', 'revenue']])
# Shows BOTH rows with order_id=1002
# order_id | customer_id | revenue
# 1002     | 202         | 80
# 1002     | 202         | 80
```

Example 2 — Intermediate: keep='first', 'last', and False

python – Code Example

```
# keep='first' – mark all duplicates except the first occurrence
print(orders_messy.duplicated(subset=['order_id'],
keep='first').sum()) # 1
# The second 1002 row is marked as duplicate

# keep='last' – mark all duplicates except the last occurrence
print(orders_messy.duplicated(subset=['order_id'], keep='last').sum())
# 1
# The first 1002 row is marked as duplicate

# keep=False – mark ALL occurrences of a duplicate
print(orders_messy.duplicated(subset=['order_id'], keep=False).sum())
# 2
# Both 1002 rows are marked

# Practical: see all duplicate records before dropping
all_dups = orders_messy[orders_messy.duplicated(subset=['order_id'],
keep=False)]
print(all_dups[['order_id', 'customer_id', 'quantity', 'revenue']])
```

Student

You said I should keep the most recent record per order. How do I do that with `drop_duplicates` ?

ChatGPT • AI Assistant

`keep='last'` keeps the last occurrence *in the current row order* of the DataFrame. To reliably keep the most recent record, you should **sort by the timestamp column first**, then `drop_duplicates(keep='last')` :

Example 3 — Practical: Keep the most recent record per key

python – Code Example

```
# Add a timestamp column to simulate update times
orders_messy['updated_at'] = pd.to_datetime([
    '2024-01-05', '2024-01-07', '2024-01-07', '2024-01-10',
    '2024-01-12', '2024-01-15', '2024-01-18',
    '2024-01-20', # This is the later version of order_id=1002
    '2024-01-20'
])

# Sort by order_id then updated_at so the latest is last in each group
orders_sorted = orders_messy.sort_values(['order_id', 'updated_at'])

# keep='last' now reliably keeps the most recent version
orders_deduped = orders_sorted.drop_duplicates(
    subset=['order_id'],
    keep='last'
).reset_index(drop=True)

print(f'Before: {len(orders_messy)} rows')
print(f'After: {len(orders_deduped)} rows')
print(orders_deduped[['order_id', 'customer_id', 'revenue',
    'updated_at']])
```

Example 4 — Real Business: Log what was dropped before removing

python – Code Example

```
# Professional pattern: always log dropped duplicates
def deduplicate_with_log(df, subset, keep='last', sort_by=None):
    """Deduplicates and returns (cleaned_df, dropped_df)."""
    if sort_by:
        df = df.sort_values(sort_by)

    # Mark rows that will be DROPPED
    keep_mapping = {'first': 'last', 'last': 'first'} # Invert logic
    for dropped_mask
        dropped_mask = df.duplicated(subset=subset, keep=keep)
        dropped = df[dropped_mask].copy()

    # Perform deduplication
    cleaned = df.drop_duplicates(subset=subset,
        keep=keep).reset_index(drop=True)
```

```

print(f'Deduplication report:')
print(f'  Original rows : {len(df)}')
print(f'  Rows removed  : {len(dropped)}')
print(f'  Remaining rows: {len(cleaned)}')
if len(dropped) > 0:
    print(f'  Dropped rows (sample):')
    print(dropped[subset].head())

return cleaned, dropped

cleaned_orders, dropped_orders = deduplicate_with_log(
    orders_messy,
    subset=['order_id'],
    keep='last',
    sort_by=['order_id', 'updated_at']
)

```

Example 5 — Advanced: Multi-column deduplication for transaction tables

python – Code Example

```

# Deduplication on composite key: customer + product + date
# (same customer cannot buy same product twice on same day – business rule)
transactions = pd.DataFrame({
    'customer_id': [201, 201, 202, 201, 202],
    'product_id': [301, 301, 302, 303, 302],
    'order_date': pd.to_datetime(['2024-01-05', '2024-01-05',
                                   '2024-01-07', '2024-01-05', '2024-01-09']),
    'amount':      [1000, 1000, 80, 75, 80]
})

# Composite key deduplication
unique_txns = transactions.drop_duplicates(
    subset=['customer_id', 'product_id', 'order_date'],
    keep='first'
)
print(f'Transactions: {len(transactions)} → {len(unique_txns)} after dedup')
# 5 → 4: the duplicate (201, 301, 2024-01-05) row is removed

```

❗ COMMON PITFALL / MISTAKE

Incorrect — dropping duplicates without inspecting first:

python – Code Example

```
df = df.drop_duplicates(subset=['order_id']) # Which one did you
keep? Did you check?
# If the first occurrence has stale data and the last has updated
data, this silently keeps stale!
```

Correct — inspect, sort, then drop:

python – Code Example

```
# 1. Inspect first
print(df[df.duplicated(subset=['order_id'], keep=False)])
# 2. Sort to control which row is 'last'
df = df.sort_values(['order_id', 'updated_at'])
# 3. Drop with intention
df = df.drop_duplicates(subset=['order_id'],
keep='last').reset_index(drop=True)
```

🌟 AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

"Act as a senior data engineer. I want to master Pandas data quality operations. Give me 10 exercises on a messy retail dataset (orders table with NaN values, type issues, and duplicates). For each exercise: describe a specific data quality problem (e.g., mixed-type column, duplicate customer records, missing revenue for specific categories), ask me to write the detection + fix code, after I attempt it show the optimal professional solution, point out any silent bugs my approach might have (leakage, wrong keep parameter, coerce vs ignore), and give one interview question. Cover: isna detection, MCAR vs MAR identification, fillna strategies, data leakage trap, pd.tonumeric, and dropduplicates with sort + log pattern. Do not continue until I answer."

SUMMARY CARD**Section 4 Key Takeaways:**

- `np.nan` is a float; `None` is a Python object; both detected by `.isna()` — never `= np.nan`
- Inserting NaN into an int column promotes it to float64 — use Pandas `Int64` to avoid this

- `.isna().sum()` for column audit; `.isna().any(axis=1)` for row detection; `missing_audit()` for full summary
- MCAR = random (safe to fill with stats); MAR = depends on other cols (group fill); MNAR = high risk (do not fill blindly)
- `dropna(subset=)` for targeted row removal; `fillna(value/method)` for replacement; `interpolate()` for smooth sequences; `ffill/bfill` for time series
- **Always split data before computing fill statistics — filling before split = data leakage**
- `pd.to_numeric(errors='coerce')` for mixed-type columns — always log coerced NaN rows
- `duplicated(subset, keep=False)` to inspect all duplicates; sort by timestamp then `drop_duplicates(keep='last')` to retain most recent
- Always log dropped duplicates in production pipelines

🔍 PRACTICE QUIZ

1. What does `df.duplicated(subset=['id'], keep=False).sum()` return for our dataset?
2. What is the correct order: sort → drop *duplicates*, or drop *duplicates* → sort?
3. What does `keep=False` in `drop_duplicates` do?
4. Predict: after `drop_duplicates(subset=['order_id'], keep='first')` on 9 rows with one duplicate `order_id`, how many rows remain?
5. You drop duplicates and later discover you removed the most recent version — which parameter was wrong?
6. Debug: `df.drop_duplicates()` removes no rows but you know `order_id` 1002 appears twice — why?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. 2 — both rows with `order_id=1002` are marked when `keep=False`
2. Sort first, then `drop_duplicates` — sorting ensures `keep='last'` retains the most recent row
3. Drops ALL occurrences of any duplicate — no occurrence of a duplicated key is kept
4. 8 rows — 9 original minus 1 duplicate occurrence removed

5. `keep='first'` was used when it should have been `keep='last'` (after sorting by `updated_at`)
6. `drop_duplicates()` with no `subset` does a full-row comparison — if the two `order_id=1002` rows differ in any other column (e.g., different `updated_at`), they are not considered exact duplicates. Fix: `drop_duplicates(subset=['order_id'])`

🔍 CHECKPOINT QUESTION

You have a 1-million-row customer table where `customer_id` should be unique, but 5,000 customers have 2 records each. You want to keep the record with the highest `account_balance`. Write the complete deduplication pipeline: sort, `drop_duplicates` with the correct keep parameter, and log what was removed.

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including missing data and deduplication, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Data Quality Issues Map for the Messy Orders Dataset:** A annotated table diagram showing the messy orders DataFrame with 9 rows and 6 c...